

Lab Experiment: 6

Title: Design and Implementation of Decision Tree Induction Algorithm for Classification

Course title: Data Mining Problem Description

Course code: CSE-452
4th Year 1st Semester Examination 2025

Date of Submission: 08/04/2026



Submitted to-

Md. Humayun Kabir

Professor

Department of Computer Science and Engineering
Jahangirnagar University

&

Dr. Md Musfique Anwar

Professor

Department of Computer Science and Engineering
Jahangirnagar University
Savar, Dhaka-1342

Class Roll	Exam Roll	Name
383	210903	Abdullah Nazmus-Sakib
379	210899	Shah Sultan
391	210911	Md Khaled Mahmud
395	210915	Tawhidul Islam

1. Objective

The objective of this experiment is to design and implement a **Decision Tree Classification Algorithm** using a dataset and analyze the classification results.

Decision Trees are used to **predict class labels** based on input features by splitting data into branches.

2. Dataset Description

The dataset contains student academic performance:

Attributes:

- FirstYear (GPA)
- SecondYear (GPA)
- ThirdYear (GPA)
- AVG_Till_Now (GPA)
- Performance_Class (Target Class)

Class Labels:

- Excellent
- Good
- Average
- Poor

Total Instances: **52**

3. CSV Dataset Creation

Dataset is prepared in Excel and saved as:

`student_performance.csv`

Then converted into ARFF format for WEKA.

4. Functions Used in Algorithm

1. connectCSV()

Purpose: Load dataset from CSV file

Arguments:

- file_path (string)

Returns:

- dataset (list)
-

2. createNode()

Purpose: Create a decision tree node

Arguments:

- attribute
 - threshold
 - left_child
 - right_child
-

3. split()

Purpose: Split dataset based on condition

Arguments:

- dataset
 - attribute
 - value
-

4. calculateGini() / calculateEntropy()

Purpose: Measure impurity

5. bestSplit()

Purpose: Find best attribute for splitting

6. buildTree()

Purpose: Recursively build decision tree

7. displayDTree()

Purpose: Display decision tree

5. Pseudocode (Group Work Division)

◆ Roll 383

Pseudocode

```
procedure createNode(attribute, threshold)
1: create new node
2: assign attribute and threshold
3: return node
```

Python Code

```
class Node:
    def __init__(self, feature=None, threshold=None, left=None, right=None,
value=None):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value
```

◆ Roll 379

Pseudocode

```
procedure split(dataset, feature, threshold)
1: left = data where feature <= threshold
2: right = data where feature > threshold
3: return left, right
```

Python Code

```
def split(dataset, feature, threshold):
    left = dataset[dataset[feature] <= threshold]
    right = dataset[dataset[feature] > threshold]
    return left, right
```

◆ Roll 391

Pseudocode

```
procedure buildTree(dataset)
1: if all data same class → return leaf
2: find best split
3: split dataset
4: recursively build left and right
5: return node
```

Python Code

```
def build_tree(dataset):
    if len(set(dataset['label'])) == 1:
        return dataset['label'].iloc[0]

    feature = 'AVG_Till_Now'
    threshold = dataset[feature].mean()

    left, right = split(dataset, feature, threshold)

    return Node(
        feature=feature,
        threshold=threshold,
        left=build_tree(left),
        right=build_tree(right)
    )
```

6. Expected Decision Tree

Based on logic:

```
AVG_Till_Now <= 3.25 → Poor
AVG_Till_Now <= 3.50 → Average
AVG_Till_Now <= 3.75 → Good
AVG_Till_Now > 3.75 → Excellent
```

7. Obtained Decision Tree (From WEKA)

```
AVG_Till_Now <= 3.492
|   AVG_Till_Now <= 3.242 → Poor
|   AVG_Till_Now > 3.242 → Average
AVG_Till_Now > 3.492
|   AVG_Till_Now <= 3.738 → Good
|   AVG_Till_Now > 3.738 → Excellent
```

8. Result Analysis

- Accuracy: **94.23%**
- Correctly Classified: **49 / 52**
- Tree Size: **7 nodes**
- Leaves: **4**

The model shows high accuracy and correctly classifies most instances.

9. Conclusion

In this experiment, a **Decision Tree classifier** was successfully implemented using both **manual logic and WEKA (J48)**.

The results show that **AVG_Till_Now** is the most important feature for classification.

Decision Trees are simple, interpretable, and effective for classification problems